IT Helpdesk Ticketing System — Project Specification

> **Status:** Source of truth for this project. Last updated: 2026-11-12. **Audience:** The project owner (a university student) and any AI coding agent working in this repository. **Location:** Save this file as `docs/PROJECT_SPEC.md`.

---

0. Rules for AI agents (read this first)

This document defines what to build and how. Follow it exactly.

Keywords. MUST / MUST NOT = hard rule. SHOULD = default unless the owner says otherwise.

0.1 Working rules

1. **Read this whole file before writing code.** Re-read the relevant section before each task.
2. **Do not invent features.** If something is not in Section 4 (Scope), it does not exist. Ask first.
3. **Ask before you:** add a dependency, change the data model (Section 8), change the tech stack (Section 2), or deviate from any rule here.
4. **Work in small steps.** One feature or one milestone (Section 15) at a time. Finish it, run the tests, then stop and report.
5. **Keep code simple and readable.** The owner is learning and must be able to explain every line in a written report and in front of a lecturer. Prefer clear code over clever code. No over-engineering, no unnecessary abstractions.
6. **Explain briefly.** After each task, report: what changed, which files, how to run it, which tests were added, and anything the owner must decide.
7. **Do not touch unrelated code.** No drive-by refactors.
8. **Never fabricate results.** Test results, timings, and screenshots in the report must come from real runs. Never invent numbers.
9. **If this document and a request conflict, stop and ask the owner.**

0.2 Quick reference

Item	Value
Language	Python
Framework	Django (server-rendered templates)
UI	Bootstrap 5, served locally from `static/`
Database	SQLite (Django default)
Package manager	`uv`
Runs on	The owner's laptop, `localhost`, opened in a desktop browser
Internet needed at runtime	No
Roles	Admin, Technician, Reporter
Notifications	In-app only

---

1. Project overview

Name: IT Helpdesk Ticketing System (web-based information system).

Problem. IT problems are reported by word of mouth or chat messages. Reports get forgotten, nobody knows who is handling what, reporters keep asking for updates, and managers cannot see which problems happen most.

Solution. A web application where:

- a **Reporter** creates a ticket for an IT problem,
- an **Admin** assigns it to a **Technician**,

- the **Technician** works on it and updates its status,
- every change is recorded, overdue tickets are clearly visible, and the Admin sees dashboards and reports.

Context. This is a university final project. A written report follows a fixed template with six chapters (Introduction, Literature Review, Analysis & Design, Implementation, Testing, Conclusion) plus appendices. The system must therefore be **testable, measurable, and well documented**.

Environment. One laptop runs the Django development server and the SQLite database. Users open the system in a browser at `http://127.0.0.1:8000`. Other computers on the same local network MAY be used for user testing by opening the laptop's IP address; this is optional.

---

2. Technology stack (fixed)

2.1 Required

Layer	Choice	Notes
Language	Python	Use a version supported by the chosen Django release
Framework	Django	Use a current stable release; pin the version in `pyproject.toml`
Database	**SQLite**	Django's default. No server, no connection setup
Templates	Django Templates	Server-rendered pages
UI toolkit	**Bootstrap 5**	Files stored locally in `static/` (offline-capable)
Packages	`uv`	`uv add <package>`, `uv run <command>`
Settings	`django-environ`	Secrets and settings from `.env`
Images	`Pillow`	Only for the ticket image attachment
Version control	Git	Small commits

2.2 Forbidden unless the owner explicitly approves

- PostgreSQL, MySQL, or any other database server.
- Frontend frameworks or build tools (React, Vue, Angular, Node/npm build steps, Tailwind build).
- Django REST Framework or any API layer.
- Celery, Redis, background workers, or schedulers.
- External services (email providers, SMS, WhatsApp, cloud storage, analytics).
- CDN links at runtime. All CSS/JS must be served from `static/`.
- Heavy JavaScript. Small vanilla JS snippets are allowed only when necessary.

2.3 Initial setup commands (reference)

```
uv init helpdesk
cd helpdesk
uv add django django-environ pillow
uv run django-admin startproject config .
uv run python manage.py startapp accounts
uv run python manage.py startapp tickets
uv run python manage.py startapp reports
uv run python manage.py migrate
uv run python manage.py runserver
```

> **Important:** Create the custom user model (Section 3.2) **before the first** `migrate`. Changing the user model later is painful.

---

3. Users and roles

3.1 Roles

Role	Description
Admin	Assigns tickets, manages master data and users, sees dashboards and reports
Technician	Works on assigned tickets, updates status, writes the solution
Reporter	Creates tickets, tracks own tickets, comments, closes own resolved tickets

3.2 Implementation

- Use a **custom user model** `accounts.User` that extends `AbstractUser` and adds a `role` field with choices `ADMIN`, `TECHNICIAN`, `REPORTER`.
- Set `AUTH_USER_MODEL = "accounts.User"` before the first migration.
- Passwords use Django's built-in hashing. Never store or log plain-text passwords.
- Permission checks MUST be done **on the server** in every view (decorator or mixin). Hiding a button in a template is not security.
- Django Admin (`/admin/`) is used for master data and user management and MUST be accessible only to the Admin role (`is_staff=True` only for Admin users).

3.3 Permission matrix

Action	Admin	Technician	Reporter
Log in / log out	✓	✓	✓
Create a ticket	✓	✓	✓
View all tickets	✓	✗	✗
View a ticket	any	only tickets assigned to them	only tickets they created
Assign / reassign a ticket	✓	✗	✗
Change status	✓ (any valid transition)	only on their assigned tickets	✗ (except closing, see 5.2)
Add a comment	✓	on tickets they can view	on tickets they can view
Manage categories, priorities, users	✓	✗	✗
View dashboard and reports	✓	own "My Tickets" only	own tickets list only

When a user requests a ticket they are not allowed to see, return **HTTP 404** (do not reveal that it exists).

---

4. Scope

4.1 In scope (core features)

ID	Feature	Who
FR-01	Login, logout, role-based access	All
FR-02	Create ticket: title, description, category, priority, optional image	Reporter (all roles may create)
FR-03	List and view own tickets with current status	Reporter
FR-04	Assign / reassign ticket to a technician	Admin
FR-05	Change ticket status; record solution note	Technician, Admin
FR-06	Comments inside a ticket	All involved
FR-07	Automatic status history	System
FR-08	Manage master data (users, categories, priorities)	Admin
FR-09	Dashboard and reports	Admin
FR-10	"My Tickets" dashboard sorted by urgency	Technician

FR-11	SLA due date, ticket age colour, overdue marker	System
FR-12	In-app notifications (badge count + list)	All
FR-13	Search and filter tickets (status, priority, category, date)	Admin, Technician

4.2 Out of scope (do NOT build; list as future work in the report)

- Email, SMS, or WhatsApp notifications
- PDF/Excel export
- Satisfaction rating after closing
- Knowledge base of solutions
- REST API
- Multi-language UI
- Deployment to a public server
- Reopening closed tickets
- Real-time updates (WebSockets)

---

5. Ticket lifecycle

5.1 Statuses

`OPEN` → `ASSIGNED` → `IN_PROGRESS` → `RESOLVED` → `CLOSED`

Status	Meaning
`OPEN`	Created, no technician yet
`ASSIGNED`	A technician has been assigned
`IN_PROGRESS`	The technician is working on it
`RESOLVED`	The technician has fixed it and written a solution note
`CLOSED`	Confirmed finished; read-only

5.2 Allowed transitions

From	To	Who	Condition
(new)	`OPEN`	Creator	On ticket creation
`OPEN`	`ASSIGNED`	Admin	Assignee chosen
`ASSIGNED`	`ASSIGNED`	Admin	Reassign to another technician
`ASSIGNED`	`IN_PROGRESS`	Assigned technician, Admin	—
`IN_PROGRESS`	`RESOLVED`	Assigned technician, Admin	`solution_note` is required
`RESOLVED`	`CLOSED`	The ticket's reporter, Admin	—

Any other transition MUST be rejected with a clear error message.

5.3 Rules

- Every status change (including creation and reassignment) MUST write a row to `TicketStatusHistory`.
- `resolved_at` is set when entering `RESOLVED`; `closed_at` when entering `CLOSED`.
- A `CLOSED` ticket is read-only: no status change, no new comments.
- Reassignment is allowed only while the ticket is `ASSIGNED` or `IN_PROGRESS`. If reassigned from `IN_PROGRESS`, the status returns to `ASSIGNED`.

---

6. SLA, ticket age, and overdue

The goal is that tickets are hard to forget.

6.1 Priorities (seed data)

Priority	Target hours
High	4
Medium	48
Low	120

6.2 Rules

- `due_at = created_at + priority.target_hours`, calculated **once when the ticket is created**.
- **Overdue** means: status is not `RESOLVED` or `CLOSED` **and** current time is later than `due_at`. Overdue is **computed**, not stored.
- **Age colour** (based on the share of target time elapsed):
  - Green: under 50%
  - Yellow: 50% up to the due time
  - Red: overdue
- **Admin dashboard panels:** "Overdue tickets" and "Unassigned for more than 1 hour".
- **Technician "My Tickets":** only their unfinished tickets, sorted by priority (High first), then oldest first.

---

7. Notifications (in-app only)

Event	Notify
Ticket assigned / reassigned	The assigned technician
Status changed	The ticket's reporter
New comment	The other participants (reporter, assignee), not the commenter

- Notifications are rows in `Notification` (Section 8).
- The navbar shows the **unread count**. A page `/notifications/` lists them; opening a notification marks it read and links to the ticket.
- Overdue warnings are **not** notification rows. They appear on dashboards (Section 6).
- Notification creation MUST live in the service layer (Section 11), not in views or templates.

---

8. Data model

Use Django models. Names are in English. All tables include sensible `__str__` methods.

8.1 Tables

`accounts.User` (extends `AbstractUser`)

Field	Type	Notes
`role`	CharField, choices	`ADMIN`, `TECHNICIAN`, `REPORTER`
(inherited)	username, first_name, last_name, password (hashed), is_active	

`tickets.Category`

Field	Type	Notes
`name`	CharField(100), unique	e.g. Network, Printer, Software, Hardware, Account/Access, Other

`tickets.Priority`

Field	Type	Notes
`name`	CharField(50), unique	High, Medium, Low
`target_hours`	PositiveIntegerField	SLA target
`sort_order`	PositiveSmallIntegerField	1 = most urgent

`tickets.Ticket`

Field	Type	Notes
`code`	CharField(20), unique	Human-readable, e.g. `HD-0001`, generated
`title`	CharField(200)	Required
`description`	TextField	Required
`category`	FK → Category, `PROTECT`	
`priority`	FK → Priority, `PROTECT`	
`status`	CharField, choices	See Section 5.1; default `OPEN`
`reporter`	FK → User, `PROTECT`, `related_name="reported_tickets"`	
`assignee`	FK → User, null/blank, `SET_NULL`, `related_name="assigned_tickets"`	Technician only
`attachment`	ImageField, optional	Max 2 MB; jpg/png
`solution_note`	TextField, blank	Required to enter `RESOLVED`
`created_at`	DateTimeField, auto	
`updated_at`	DateTimeField, auto	
`due_at`	DateTimeField	Set once at creation
`resolved_at`	DateTimeField, null	
`closed_at`	DateTimeField, null	

`tickets.TicketComment`

Field	Type	Notes
`ticket`	FK → Ticket, `CASCADE`, `related_name="comments"`	
`author`	FK → User, `PROTECT`	
`body`	TextField	Required
`created_at`	DateTimeField, auto	

`tickets.TicketStatusHistory`

Field	Type	Notes
`ticket`	FK → Ticket, `CASCADE`, `related_name="history"`	
`old_status`	CharField, blank	Empty on creation
`new_status`	CharField	
`changed_by`	FK → User, `PROTECT`	
`changed_at`	DateTimeField, auto	
`note`	CharField(255), blank	e.g. "Reassigned to Technician X"

`tickets.Notification`

Field	Type	Notes
`user`	FK → User, `CASCADE`, `related_name="notifications"`	Recipient

`ticket`	FK → Ticket, `CASCADE`	
`message`	CharField(255)	
`is_read`	BooleanField, default False	
`created_at`	DateTimeField, auto	

8.2 Relationships (for the ERD)

- User 1—N Ticket (as reporter)
- User 1—N Ticket (as assignee)
- Category 1—N Ticket
- Priority 1—N Ticket
- Ticket 1—N TicketComment
- Ticket 1—N TicketStatusHistory
- User 1—N TicketComment
- User 1—N Notification
- Ticket 1—N Notification

8.3 Data rules

- Never delete tickets, history, or comments through the UI. Deactivate users (`is_active=False`) instead of deleting them.
- Always use the Django ORM. Raw SQL is not allowed unless the owner approves; if used, it MUST be parameterized.
- Migrations are committed to Git. Never edit a migration that has already been applied; create a new one.

---

9. Pages and routes

Route	Purpose	Access
`/login/` , `/logout/`	Authentication	Public / logged in
`/`	Redirect to the role's dashboard	Logged in
`/dashboard/`	Role-based dashboard (Admin: stats and warning panels; Technician: My Tickets; Reporter: own tickets)	Logged in
`/tickets/`	Ticket list with search and filters (scoped by role)	Logged in
`/tickets/new/`	Create ticket	Logged in
`/tickets/<code>/`	Ticket detail: info, comments, history, actions	Allowed users (Section 3.3)
`/tickets/<code>/assign/`	Assign / reassign (POST)	Admin
`/tickets/<code>/status/`	Change status (POST)	Per Section 5.2
`/tickets/<code>/comment/`	Add comment (POST)	Allowed users
`/notifications/`	Notification list	Logged in
`/reports/`	Reports (Section 10)	Admin
`/admin/`	Master data and users (Django Admin)	Admin

Use **POST** for every action that changes data. Every form MUST include `{% csrf_token %}` .

---

10. Dashboard and reports (Admin)

Dashboard

- Ticket counts by status
- Overdue tickets (list)
- Unassigned for more than 1 hour (list)

Reports (with optional date range)

- Average resolution time per priority (`resolved_at − created_at`)
- Number of tickets per category
- Tickets handled and average resolution time per technician
- Percentage of tickets that were overdue when resolved

Reports are shown on screen (tables; simple Bootstrap bars are fine). No export in this version.

---

11. Project structure and coding rules

11.1 Folder structure

```
helpdesk/
├── config/                # settings, urls, wsgi
├── accounts/              # custom User, login/logout, role checks
├── tickets/               # Ticket, comments, history, notifications
│   ├── models.py
│   ├── services.py        # business rules (transitions, SLA, notifications)
│   ├── views.py
│   ├── forms.py
│   ├── urls.py
│   ├── tests/
│   └── management/commands/seed_demo.py
├── reports/               # dashboard and report views
├── templates/
│   ├── base.html          # navbar, messages, notification badge
│   ├── accounts/
│   ├── tickets/
│   └── reports/
├── static/
│   ├── bootstrap/         # local Bootstrap CSS/JS
│   └── css/app.css
├── media/                 # uploaded images (git-ignored)
├── docs/
│   ├── PROJECT_SPEC.md    # this file
│   └── TEST_LOG.md        # real test results (Section 13)
├── .env                   # git-ignored
├── .gitignore
├── manage.py
└── pyproject.toml
```

`db.sqlite3` MUST be in `.gitignore`.

11.2 Coding rules

1. **Business logic goes in `tickets/services.py`**: status transitions, assignment, SLA calculation, history writing, notification creation. Views stay thin and templates contain no business rules.
2. Use **function-based views** for readability, unless the owner asks otherwise. Keep the style consistent.
3. Check permissions at the start of every view. Use one shared helper/decorator for role checks so rules are not duplicated.
4. Follow PEP 8. Use clear names. Add short docstrings to services and non-obvious functions. Comments explain *why*, not *what*.
5. Use `select_related` / `prefetch_related` on list pages to avoid N+1 queries.
6. All templates extend `base.html` and use Bootstrap components (navbar, cards, tables, forms, badges, alerts). Use Django `messages` for feedback.
7. Settings: `TIME_ZONE = "Asia/Dili"`, `USE_TZ = True`, `DEBUG` and `SECRET_KEY` from `.env`, `ALLOWED_HOSTS` limited to local addresses.
8. Identifiers and UI text are in **English**.
9. One feature per Git commit with a clear message (e.g. `feat(tickets): add assign ticket view`).

---

12. Security rules

- Authentication required on every page except `/login/`.

- **No object-level leaks:** a reporter MUST NOT open another reporter's ticket by editing the URL; a technician MUST NOT open tickets not assigned to them (return 404).
- All state-changing requests use POST with CSRF protection.
- Never use `mark_safe` or `|safe` on user-provided content.
- Validate uploads: images only (jpg/png), maximum 2 MB. Store under `media/` with a generated filename.
- No secrets in Git. Keep `.env` out of the repository and provide `.env.example`.
- Use only dummy data in seed data, screenshots, and the report. No real personal data.
- Log or record who changed what (this is what `TicketStatusHistory` is for).

---

13. Testing requirements

Use Django `TestCase`. Run all tests with:

```
uv run python manage.py test
```

13.1 Minimum test cases

ID	Test	Expected
T-01	Login with valid credentials	Redirect to dashboard
T-02	Login with wrong password	Error message, no session
T-03	Anonymous user opens `/tickets/`	Redirect to login
T-04	Create ticket with valid data	Ticket saved as `OPEN`, `due_at` set, history row created
T-05	Create ticket with empty title	Form error, nothing saved
T-06	Reporter opens another reporter's ticket by URL	404
T-07	Technician opens a ticket not assigned to them	404
T-08	Technician tries to assign a ticket	Forbidden
T-09	Admin assigns a ticket to a technician	Status `ASSIGNED`, history row, notification for the technician
T-10	Technician changes `ASSIGNED` → `IN_PROGRESS`	Allowed, history row
T-11	Invalid transition (e.g. `OPEN` → `RESOLVED`)	Rejected with message
T-12	Resolve without `solution_note`	Rejected
T-13	Reporter closes own `RESOLVED` ticket	`CLOSED`, `closed_at` set
T-14	Comment on a `CLOSED` ticket	Rejected
T-15	SLA: High priority ticket	`due_at = created_at + 4 hours`
T-16	Overdue calculation	Overdue only when unfinished and past `due_at`
T-17	Notification created on assign and on status change	Rows exist for the correct users
T-18	Reporter requests `/reports/`	Forbidden

13.2 Test log

Keep `docs/TEST_LOG.md` with **real** results: date, test ID, result (Pass/Fail), notes, and for user acceptance testing the real measured times (manual way vs. using the system). **Never write results that were not actually observed.**

---

14. Seed (demo) data

Provide a management command:

```
uv run python manage.py seed_demo
```

It MUST be safe to run more than once and create **dummy data only**:

- 1 Admin, 3 Technicians, 8 Reporters (fictional names; simple development-only passwords)
- Categories: Network, Printer, Software, Hardware, Account/Access, Other
- Priorities: High (4 h), Medium (48 h), Low (120 h)
- About 20 sample tickets in different statuses, including some overdue and some unassigned

This allows a live demo to start immediately with realistic data.

---

15. Build order (milestones)

Finish each milestone fully (code + tests + commit) before starting the next.

#	Milestone	Covers
M0	Project setup, settings, `.gitignore`, `base.html`, local Bootstrap	—
M1	Custom user, login/logout, role checks	FR-01
M2	Models, migrations, Django Admin for master data and users	FR-08
M3	Create ticket, ticket list, ticket detail	FR-02, FR-03
M4	Assign, status transitions, history, solution note	FR-04, FR-05, FR-07
M5	Comments	FR-06
M6	SLA, age colours, overdue, technician "My Tickets", admin warning panels	FR-10, FR-11
M7	In-app notifications	FR-12
M8	Search and filter, dashboard, reports	FR-13, FR-09
M9	Seed command, full test run, polish, screenshots	—

---

16. Definition of done

A task or milestone is done only when:

- [ ] The feature works as described in this document
- [ ] Permission rules (Section 3.3) are enforced on the server
- [ ] Tests for the feature exist and pass (`manage.py test`)
- [ ] No new dependency was added without owner approval
- [ ] Code is simple and readable, with short docstrings where needed
- [ ] Migrations are created and committed
- [ ] The agent has reported what changed and how to run it

---

17. Future work (not part of this version)

Email/WhatsApp notifications, PDF/Excel export, satisfaction rating, knowledge base, migration to PostgreSQL for many concurrent users, REST API, multi-language UI.

Known limitation to state honestly in the report: SQLite is enough for one laptop and a small group of test users, but it is not designed for many users writing at the same time.

---

18. Notes for the written report

The agent should help the owner keep material for the report as work progresses:

- Keep a short decision log (what was decided and why).

- Remind the owner to capture screenshots of: login, dashboards, ticket form, ticket detail, reports.
- Measurable results for the testing chapter (collect from real use only):
  - Time to find a ticket's status: manual way vs. the system
  - Average resolution time per priority
  - Percentage of overdue tickets before and after using the system
  - Security check: a reporter cannot open another reporter's ticket by URL